FRI

Computing with DNA Strand Displacement

Andrej Jočić

Master´s study programme Computer and Information Science, study field Computer and Information Science

Faculty of Computer and Information Science

Master's thesis

Supervisor: prof. dr. Miha Moškon
Cosupervisor: asist. Ana Halužan Vasle

Copyright: This work is licensed under a CC-BY-4.0 license. The software and results produced in this work are released under GPL-3.0-or-later at https://github.com/andrejjocic/displacement-cascade-compiler.

Verifying the correctness of DNA strand displacement cascades that implement synthetic biological neural network classifiers (winner-take-all, loser-take-all) built on the seesaw architecture is experimentally costly, and existing modelling tools either lack the flexibility to explore translation-scheme parameters or are not fit for modelling the pairwise annihilation reactions present in these circuits. We developed DISCO (DISplacement cascade COmpiler), a pipeline for assembling signal-processing circuits using a modular, parametrized translation scheme that enables systematic *in silico* validation and verification of design variants through reaction enumeration (Peppercorn), simulation, and checking formal equivalence to a high-level specification. We derived the timescale-separation parameters required for correct annihilation modelling and, since these inevitably create transient complexes that dominate verification complexity in larger circuits, devised a local-toehold assignment scheme that substantially reduces this complexity without significantly affecting simulation results.

DNA strand displacement | synthetic biological neural networks | winner-take-all classifier | seesaw circuits | translation scheme | formal verification

1. Introduction

Evolution has produced astonishingly complex biomolecular machines, and engineering comparable ones could enable precise diagnostics and treatment, even combined, without human intervention: computational theranostics inside living cells [1]. On the way there we must learn to induce precisely prescribed computational behaviour in a molecular circuit in solution, i.e. to *program molecules* [2]. Because experimental debugging of molecular programs is slow and expensive, we rely on mathematical analysis, modelling and simulation. The goal of this work is a tool for *in silico* testing and verification of pattern-recognition circuits built from synthetic DNA components [3, 4]. Such classifiers are promising for biomarker recognition in diagnostics and for detecting harmful substances in the environment.

1.1. DNA strand displacement. Early DNA computing aimed to outrun silicon by massive parallelism [5, 6], but it did not scale to practical problem sizes. Recent work therefore targets embedded controllers that operate autonomously and reliably in biological environments, where speed is secondary [3, 7, 8]. Among engineered mechanisms for processing biochemical signals, DNA strand displacement cascades have proved particularly promising [9]. They are theoretically universal [10], run autonomously without enzymes, and have been used to build an oscillator [11] and a square-root circuit [12].

The key step is *branch migration*, in which two copies of a domain compete to hybridise with a complementary domain, and a three-way branch performs a random walk [13] (Figure 1). This is unlikely unless the strands are already joined via a neighbouring domain. Hence a double-stranded complex is given a single-stranded *toehold* domain to which an invading strand can bind. A reaction that begins with stable binding to a toehold is called *toehold-mediated strand displacement* (TMSD) (Figure 2) [14]; its rate is controlled by the toehold length and grows exponentially with it [15].

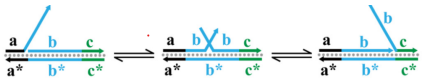


Figure 1. Three-way branch migration across domain b. Adapted from [9].

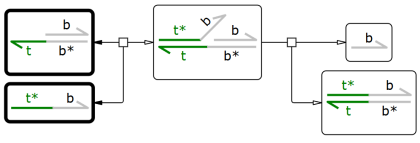


Figure 2. State diagram showing displacement of strand b via toehold t. The double-stranded complex and the invading strand are outlined in bold. Initial binding is reversible, while displacement is irreversible in this case, since no toehold remains exposed in the final state.

Računanje z izpodrivanjem verig DNA

Preverjanje pravilnosti kaskad izpodrivanja verig DNA, ki implementirajo sinteznobiološke klasifikatorje (zmagovalec-vzame-vse, poraženec-vzame-vse) z arhitekturo seesaw, je eksperimentalno zahtevno, obstoječa modelirna orodja pa so bodisi premalo prilagodljiva za raziskovanje parametrov prevajalne sheme bodisi neprimerna za modeliranje reakcij parnega izničevanja, ki nastopajo v teh vezjih. Razvili smo orodje DISCO (DISplacement cascade COmpiler), cevovod za sestavljanje vezij za procesiranje signalov z modularno, parametrizirano prevajalno shemo. Orodje omogoča sistematično *in silico* validacijo in formalno verifikacijo različic sheme s pomočjo naštevanja reakcij (Peppercorn), simulacije in preverjanja formalne ekvivalence s specifikacijo. Izpeljali smo parametre ločevanja časovnih skal, potrebne za pravilno modeliranje izničevanja. Ker ti parametri neizogibno ustvarijo prehodne komplekse, ki pri večjih vezjih prevladujejo v verifikacijski zahtevnosti, smo zasnovali shemo dodelitve lokalnih domen toehold, ki to zahtevnost bistveno zmanjša brez znatnega vpliva na simulacijske rezultate.

izpodrivanje verig DNA | sinteznobiološke nevronske mreže | klasifikator zmagovalec-vzame-vse | vezja seesaw | prevajalna shema | formalna verifikacija

1.2. Molecular programming and domain-level systems. A chemical reaction network (CRN) is a set of reactions $\sum_i \alpha_i R_i \xrightarrow{k} \sum_j \beta_j P_j$, where R_i are reactants, P_j products, and α_i, β_j stoichiometric coefficients (almost always 1 for TMSD). A monomolecular reaction $R_1 \xrightarrow{k} \sum_j P_j$ has rate $k[R_1]$ where rate constant k has units s^{-1}. A bimolecular reaction $R_1 + R_2 \xrightarrow{k} \sum_j P_j$ has rate $k[R_1][R_2]$ where k is measured in $M^{-1}s^{-1}$. Rates of reactions with more than two reactants are negligible in the context of TMSD.

Aside from modelling existing systems, CRNs can also serve as a programming language: a *formal CRN* prescribes chemical processes and says nothing about physical implementation [16, 17]. Formal reactions need not obey mass conservation, e.g. $A + B \to \varnothing$; the implementing reaction pathway must then end in inert *waste species* [18], which for TMSD are those with no exposed toehold. Rate constants are optional in a formal CRN and are prescribed only when relative reaction speeds matter.

A *translation scheme* maps each formal species to one or more implementation species, called *signal species*, and defines supporting *fuel species* that act as catalysts. Its standard output is a *domain-level strand displacement (DSD) system* [16]. Its basic elements are *domains*, nucleotide sequences that act as a unit in hybridisation. The complement of domain d is d^*. Domains are conceptually divided into longer *migration domains* and shorter toeholds. A *strand* is a directed list of domains, written $\langle a\ b\ c \rangle$ from the $5'$ end to the $3'$ end. Domain-level modelling assumes that well-designed, orthogonal nucleotide sequences will be assigned to domains, e.g. using NUPACK [19].

A *complex* is a particular secondary structure formed within one strand or across several strands [20]. Formally, a secondary structure of strands $S = \{s_1, \ldots, s_n\}$ is a symmetric map $\pi\colon (S \times \mathbb{N}) \to (S \times \mathbb{N}) \cup \emptyset$, where $\pi(s_i, j) = (s_l, k)$ means the j-th domain of s_i is bound to the k-th domain of s_l, and $d \mapsto \emptyset$ denotes a free domain d.

The dynamics of a DSD system are defined by interaction rules: bimolecular binding of free complementary domains, unbinding (often allowed only for short domains, up to about 7 nt), monomolecular binding within a complex, and three-way (or four-way) branch migration [20]. TMSD is thus two elementary steps: bimolecular toehold binding followed by branch migration.

1.3. Verification and undesired reactions. We have explicit control only over the shapes of molecules, not over all their possible interactions. Computational (*in silico*) verification aims to find bugs or to show that side reactions are negligible. Verification of a translation scheme begins with systematic computation of all possible reactions among the implementation species, by a *reaction enumerator*, giving an *implementation CRN*, which is then checked for equivalence with the formal CRN [16] (Figure 3). Equivalence can be checked informally by simulation, or formally, which is an open research problem [17, 18, 21]. The open-source library `crnverifier`[1] offers pathway decomposition equivalence [18], bisimulation equivalence [17], and two hybrids of the two [18]. None of them accounts for rate constants. False negatives can occur, where an implementation CRN is correct in practice, but its equivalence to the formal CRN is not provable.

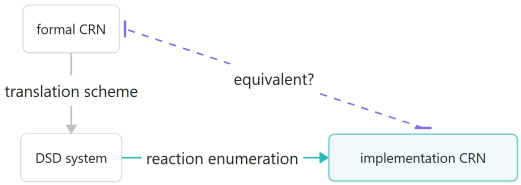


Figure 3. Verification of a translation scheme implementation. CRNs are used for programming (top) and for modelling (right).

Which reactions count as bugs depends on the scheme. Here we treat *remote toehold-mediated strand displacement* (RTMSD) [22] as undesired, as well as *toehold occlusion*: a signal strand binds a toehold of a mismatched double-stranded molecule, so binding cannot proceed to branch migration and is *unproductive* (Figure 4). At low concentrations occlusion is negligible when the toehold is short, since it quickly unbinds [7].

1.4. Synthetic biological classifiers. We use the shallow neural classifier of Cherry and Qian [3], later extended by Rodriguez *et al.* [4], as a case study. Its building block is the *seesaw gate*: an input signal strand displaces an output signal strand from a gate, reversibly. Two seesaw reactions plus fuel form a catalytic cycle that amplifies signals; signal-propagation networks are called *seesaw circuits* [7].

[1] https://github.com/DNA-and-Natural-Algorithms-Group/crnverifier

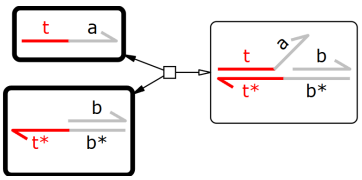


Figure 4. Occlusion of toehold t. The mismatched invading strand $\langle t\ a\rangle$ (top left) binds the double-stranded molecule (bottom left) unproductively, because a is not complementary to b^*. While bound, TMSD by matching strands $\langle t\ b\rangle$ is blocked.

WTA. A pre-trained non-negative weight matrix $\boldsymbol{W} \in \mathbb{R}_+^{n\times m}$ is compiled into a single fully connected layer classifying a binary vector $\boldsymbol{x} \in \{0,1\}^n$ into one of m classes. A value $a \in \mathbb{R}^+$ is represented by the relative concentration of species A (if standard concentration is 100 nM, then $[A] = 50$ nM represents $a = 0.5$). The class maximising $s_j = \sum_i w_{ij}x_i$ is output via the *winner-take-all* (WTA) function $o_j = [\![\forall k \neq j : s_j > s_k]\!]$[2]. Weighted sums are computed by formal reactions $X_i + W_{ij} + XF_i \to X_i + P_{ij}$ followed by $P_{ij} \to S_j$, with weights normalised so that $s_j \in [0,1]$. The maximum is found by *pairwise annihilation* $S_j + S_k \to \varnothing$, with annihilator fuels Anh_{jk} that decay to waste via *cooperative hybridisation*. Since the winner may be weak, *signal restoration* $S_j \to O_j$ amplifies it using a catalytic cycle. Annihilation and restoration share reactants, so annihilators have longer toeholds to enforce correct reaction order. The restoration toehold is by necessity a prefix of the annihilation toehold. Finally, *reporter gates* release fluorescent products $Fluor_j$ [3].

LTA. *Negative classification*, i.e. finding the class to which an input does *not* belong ($\arg\min \boldsymbol{x}^T\boldsymbol{W}$), is easier to realise experimentally and suits medical screening. The *loser-take-all* (LTA) function is $o_j = [\![\forall k \neq j : p_j < p_k]\!]$. It can be obtained by running WTA on the transformed weights $\boldsymbol{W}' = \frac{1}{m-1}\boldsymbol{W}(\boldsymbol{E} - \boldsymbol{I})$, which densifies the matrix. Rodriguez *et al.* [4] instead introduced *signal reversal*, implemented by irreversible seesaw reactions $\forall i \neq j : P_i + OG_{ij} \to R_j$, giving $[R_j]_\infty = \frac{1}{m-1}\sum_{i\neq j}[P_i]_0$. The price is that differences between signals shrink by a factor $m-1$.The reversed signal R_j is represented by all species $\langle r_j\ T\ s\ p_i\rangle$, $i \neq j$, where p_i is an inert *history domain* [16]. We denote the set of all species representing signal R_j using structure notation $\langle r_j\ T\ s\ ?\rangle$, similar to regular expressions.

1.5. Existing tools, motivation and goal. `Visual DSD` [23] has the most extensive functionality (bottom-up enumeration, simulation, SBML export [24]). Specialised top-down compilers exist for seesaw circuits and for WTA (Qian and Winfree[3]; Cherry[4]): they skip DSD systems and explicit enumeration, instead approximating the implementation CRN with architecture-specific rules, which may miss some undesired reactions.

The `Peppercorn` reaction enumerator [20] enables bottom-up enumeration of arbitrary DSD systems. The seesaw compiler's circuit representation format cannot express cooperative hybridisation, so the classifier cannot be given to the enumerator `Peppercorn` in that format. `Peppercorn` models a wide range of reactions, approximates rate constants from domain lengths, and applies *max-helix semantics*, propagating reactions to the maximal run of adjacent domains. It enumerates in a pseudoknot-free space of secondary structures. To limit combinatorial explosion it uses *timescale separation*: bimolecular reactions are slow and monomolecular ones fast by default, or, in a rate-dependent variant, monomolecular reactions are split by thresholds k_{slow} and k_{fast} into negligible ($k < k_{slow}$, discarded), slow ($k_{slow} \leq k < k_{fast}$), and fast ($k \geq k_{fast}$). Complexes are classified as *resting* or *transient* (resting complexes lie in strongly connected components of the fast monomolecular reaction graph with no outgoing fast reactions), and slow reactions are enumerated only from resting complexes. *CRN condensation* then merges resting complexes into *resting macrostates* and replaces fast paths through transient states by single reactions, giving a *condensed CRN*. Condensation can discard toehold occlusion when unbinding is fast.

The enumerator in `Visual DSD` models pairwise annihilation incorrectly: because compound toeholds $\langle s\ T\rangle$ on annihilators are occluded irreversibly, the circuit fails. After binding s, the enumerator applies the *toehold covering* rule, which is irreversible [25] (Figure 5). Consequently, occlusion of $\langle s\ T\rangle$ is not recognised as unproductive and thus can't be filtered by any of the enumeration semantics settings offered by `Visual DSD`. `Peppercorn`'s max-helix rule treats binding of both domains as one reaction, avoiding this issue.

[2] $[\![\cdot]\!]$ denotes Iverson brackets.
[3] https://qianlab.caltech.edu/SeesawCompiler
[4] https://www.qianlab.caltech.edu/WTAcompiler

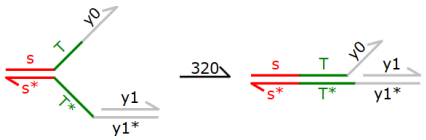


Figure 5. Covering of domain T after prior binding of s (enumerated using directive `compilation detailed`). This is the second step of occlusion of the compound toehold $\langle s\ T \rangle$. A similar reaction covers s after binding T. Both are irreversible.

We therefore present DISCO (*DISplacement cascade COmpiler*), a tool for systematic *in silico* testing, debugging and verification of arbitrary cascades of operations in the chosen synthetic biological classifiers, and for easy modification and optimisation of the translation scheme. The tool enables bottom-up testing: first individual components, then complete WTA and LTA circuits.

2. The DISCO verification pipeline

DISCO[5] takes a description of the circuit topology, i.e. the data flow between computational operations, represented by modules that include parameters for their translation into a DSD system. The first stage translates it into PIL format (used by `Peppercorn`) [20], from which the implementation CRN is enumerated; this is followed by simulation or by equivalence checking against a formal CRN assembled from the topology (Figure 6).

2.1. Implementation of the translation scheme. The PIL generator implements a parametrised translation scheme; parameters set, e.g., weight-matrix dimensions, domain lengths, relative concentrations of supporting species, or reversibility of some reactions, and metadata for later stages is added to the PIL file. The circuit is an instance of class `DSDCircuit` storing the relative concentrations of all supporting species; input concentrations are set at simulation time.

A circuit consists of *modules*, internal representations of sets of formal reactions, each describing one cascade step (e.g. a pairwise annihilation module represents reactions $\forall i < j : C_i + C_j \to \varnothing$). The partial translation scheme lives in `Module.compile`, which adds supporting species (e.g. $\forall i < j : Anh_{ij}$) at prescribed concentrations. Modules are linked by input and output instances of class `Signal`, which represent n-dimensional signals; `Signal("r", n)` acts as a generator of migration domains $S_i^{(r)} = r_i$. Domains and secondary structures are handled with the `dsdobjects`[6] library, which prevents some bugs, e.g. two different domains with the same name. Listing 1 shows assembly of a WTA circuit.

Listing 1: Assembling a DSD circuit computing WTA

```
circ = DSDCircuit()
competitor = circ.new_signal("c", dimension=n)
binary_out = circ.new_signal("o", dimension=n)
cascade = [
    PairwiseAnnihilation(input=competitor),
    SignalRestoration(input=competitor, output=binary_out),
    Reporting(input=binary_out)
]
circ.add_modules(cascade)
```

Unlike the WTA scheme [3], where annihilators have a separate toehold extension for each signal, DISCO uses a universal extension as in the LTA scheme [4]. This permits occlusion but is experimentally more robust, since correct pairwise annihilation requires equal binding rate constants of the signals to all annihilators [3]. This choice also simplifies translation scheme implementation. Rodriguez *et al.* [4] gave no domain-level design of molecules W_{ki} suitable for combining weight multiplication with signal reversal; we modified the WTA weight species [3] to be compatible with the reversal gates (Figure 7).

Seesaw circuits originally use a universal (global) toehold for scalability, since it is hard to design many orthogonal short sequences [7]. This can cause many occlusions and hence costlier enumeration and simulation. To avoid relying solely on CRN condensation, we implemented a scheme assigning several different toeholds to the circuit's species. Modules linked by a signal must have matching toeholds, otherwise the signal cannot propagate: a seesaw gate has an exposed toehold t_v binding the input and a covered toehold t_i that is part of the bound output signal [7]. Each signal $S^{(x)}$ is therefore given its own "local" toehold t_x. More distinct toeholds mean fewer occlusions and a smaller implementation CRN. The same toehold can still be reused for different signals ($t_a = t_b$ for $S^{(a)} \neq S^{(b)}$). We avoid assigning different toeholds to different strands belonging

[5] https://github.com/andrejjocic/displacement-cascade-compiler
[6] https://pypi.org/project/dsdobjects/

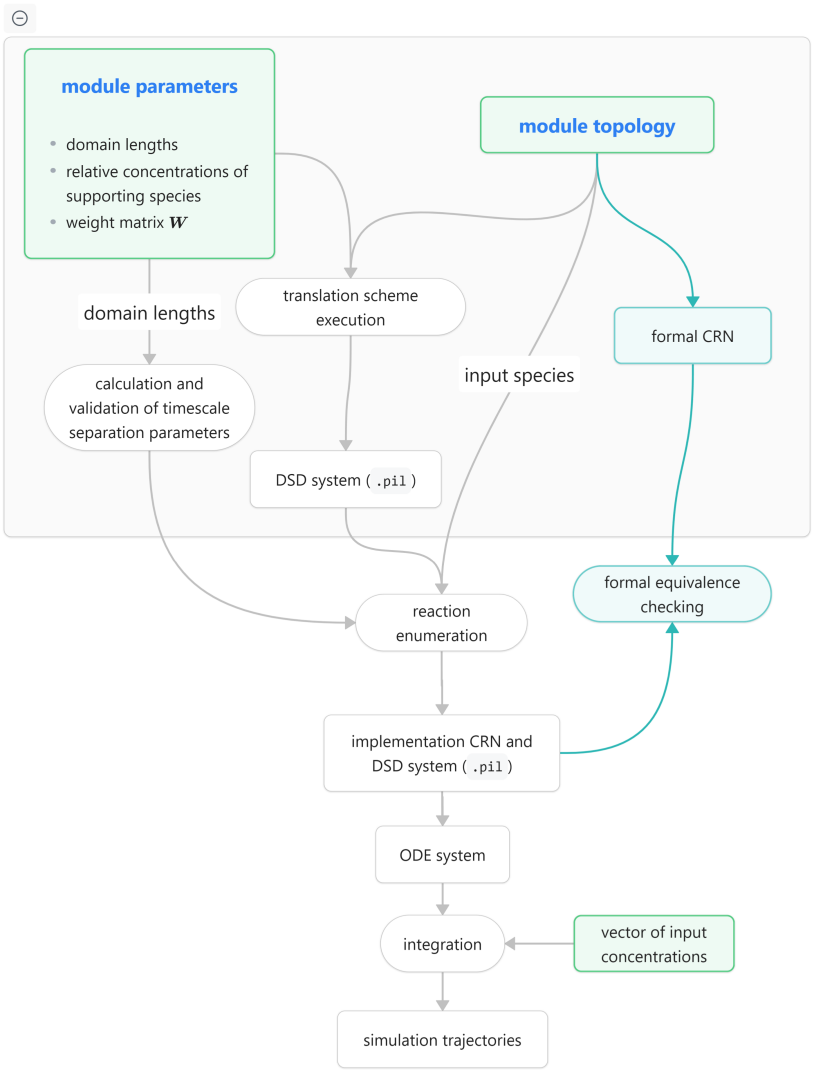


Figure 6. The DISCO pipeline. The grey outline marks our programming contribution (excluding the design of translation schemes). Green marks user input; blue marks the part enabling formal verification.

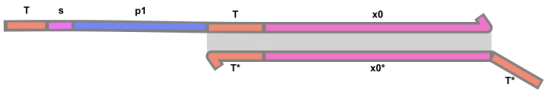


Figure 7. Structure of molecule $W_{0,1}$ for combining weight multiplication with signal reversal. Compared with WTA weight molecules [3], only the single-stranded segment $\langle T\ s\ p_1 \rangle$ on the left is changed. The compound toehold $\langle T\ s \rangle$ is identical on every W_{ki} because a universal extension s is used.

to the same signal, since correct computation often needs equal binding rate constants, which in practice requires equal DNA sequences (not merely equal lengths).

2.2. Reaction enumeration.

Initial conditions. `Peppercorn` computes only reactions reachable from the initial species, i.e. those given a non-zero concentration in the PIL file. Besides supporting species, all input species of the first module must be marked initial, so the enumerator models all possible input combinations. For most modules the i-th input strand that triggers TMSD is $\langle S_i^{(v)}\ t_v \rangle$. The list of input species is added to the PIL file as a comment so that it is available at enumeration time.

Timescale-separation parameters. Choosing the parameters requires the length of the toehold t_k that starts cooperative hybridisation for annihilation. Thanks to max-helix semantics t_k can be treated as one domain of length $|t_k|$, the sum of its subdomains' lengths.

To derive suitable parameters, we conceptually split annihilation $C_i + C_j \to \varnothing$ into two phases, each being one signal's contribution to breaking the annihilator Anh_{ij} into waste. Each phase consists of bimolecular binding followed by branch migration (Figure 8). The first phase is fully reversible via monomolecular steps: unbinding of t_k and reverse migration across c_i.

The default timescale separation by number of reactants is unsuitable, since the inverse steps of the first phase are monomolecular while the start of the second is bimolecular: fast monomolecular reactions always happen before the slow bimolecular one. We therefore need rate-dependent separation with thresholds such that at least one inverse reaction of the first phase is slow, yet none is negligible, since the process must be reversible in the absence of C_j. To keep the state space small we want as many fast reactions as possible, so we only mark unbinding of t_k as slow (impossibility of alternatives is discussed in Section 5.2). The thresholds must satisfy $k_{slow} \leq k_u(|t_k|) < k_{fast} \leq k_m(|c_i|)$, where k_u, k_m approximate the unbinding and migration rate constants and $|c_i|$ is the length of the annihilator domains across which migration occurs. Figure 9 shows this is feasible for $|t_k| \geq 5$ (the original design has $|t_k| = 7$ [4]).

DISCO sets k_{fast} as small as possible by default to limit the state space: $k_{fast} = k_u(|t_k|) + \varepsilon$ for a small constant ε. Since k_u is decreasing, unbinding of every shorter toehold $|t| < |t_k|$ is fast, so its occlusions are removed by condensation. Products of binding t_k are resting complexes because unbinding is slow, so occlusions of t_k cannot be condensed away. No toehold's unbinding may be negligible, or the occlusion becomes irreversible and cannot be removed even by condensation.

With rate-dependent separation `Peppercorn` does not distinguish toeholds from migration domains by length, so any hybridised pair of migration domains could unbind spontaneously, which is particularly problematic for reporter complexes (spontaneous dissociation causes noisy fluorescence). To forbid unbinding of migration domains d, we must set $k_{slow} > k_u(|d|)$. Since all toeholds in a well-defined DSD system are shorter than migration domains, an appropriate default is $k_{slow} = k_u(t_{max}) - \varepsilon$, where $t_{max} = |t_k|$ is the longest toehold. To label fewer monomolecular reactions as negligible, one may instead set k_{slow} as low as possible: $k_{slow} = k_u(d_{min}) + \varepsilon$ with being d_{min} the shortest migration domain. For typical lengths $k_u(|d|) \ll k_m(|d|)$ (Figure 9), so $k_{slow} \approx k_u(t_{max})$ discards no branch migration in a seesaw circuit. Without annihilation the choice of k_{slow} is free, but we must check before enumeration that no migration is discarded. Each module therefore records in metadata the distance (in monomers) across which migration occurs; by max-helix this is generally a run of several domains, e.g. $\langle T\ s\ p_i \rangle$ in signal reversal (Figure 10). Since k_m is decreasing, it suffices that $k_m(z_{max}) \geq k_{slow}$, where z_{max} is the longest migration distance in the circuit.

2.3. Formal verification of the translation. Pathway decomposition equivalence is unsuitable for translation schemes with history domains [18], so we did not use it. All other methods in `crnverifier` require, besides the formal and implementation CRNs, an initial partial interpretation, i.e. a mapping from implementation species to formal species.

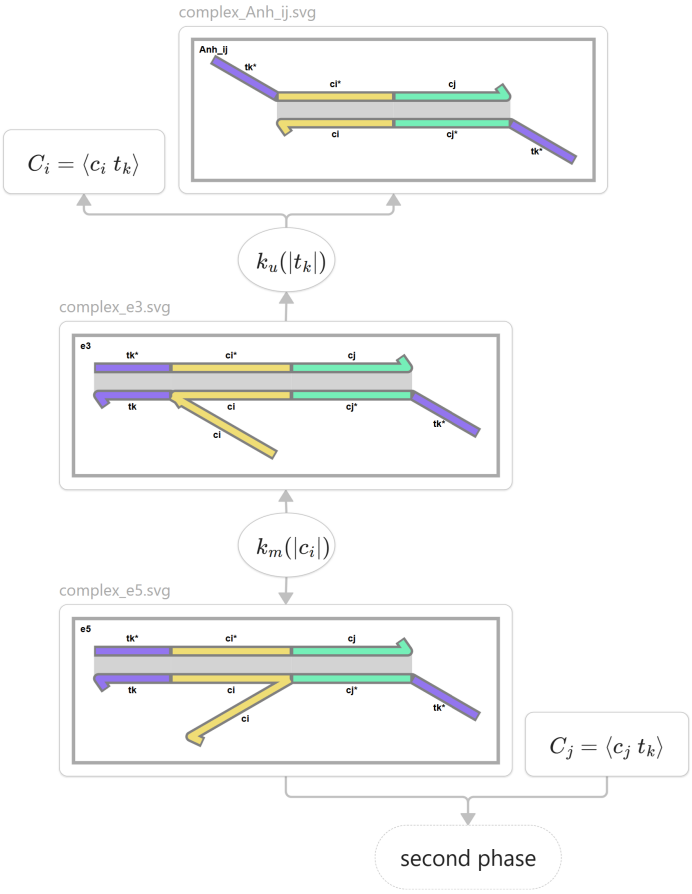


Figure 8. First phase of pairwise annihilation $C_i + C_j + Anh_{ij} \rightarrow \varnothing$. Rectangular cells are species; round cells are reactions with rate constants approximated by k_u (unbinding) and k_m (migration). States are arranged chronologically from top to bottom, so inverse reactions go upwards. Binding of t_k between C_i and Anh_{ij} is not shown; its inverse (unbinding) is. At the bottom, complex e5 binds the second signal C_j via a free copy of t_k, starting the second phase, which proceeds like the first.

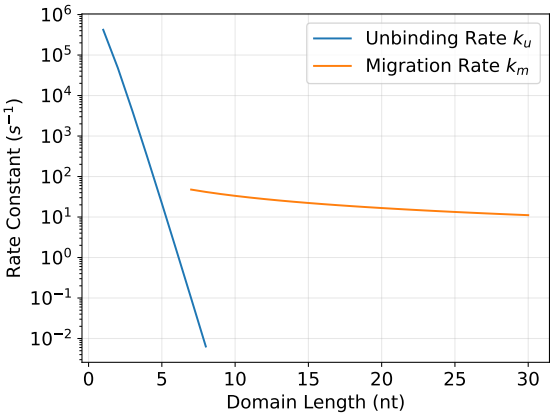


Figure 9. Rate-constant approximation computed by `Peppercorn`. Blue: unbinding constants $k_u(\ell)$, $\ell \in [1, 8]$. Orange: constants of three-way branch migration $k_m(\ell)$, $\ell \in [7, 30]$.

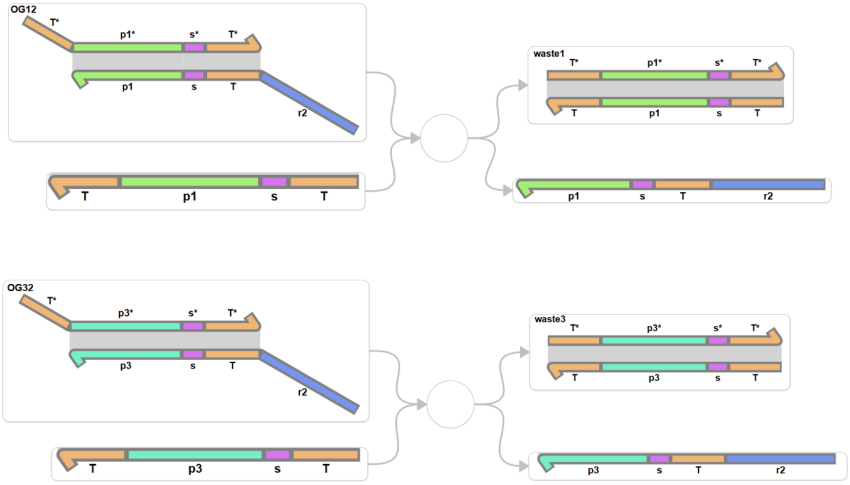


Figure 10. DSD implementation of signal reversal of $m = 3$ signals. Top: implementation of $OG_{12} + P_1 \to R_2$. Bottom: implementation of $OG_{32} + P_3 \to R_2$. Both single-stranded products together represent reversed signal R_2. Strands are folded at the 3′ end. Domain s extends toehold T.

For each module we implemented a generator of the complexes that together represent formal signal $S_i^{(o)}$, where $S^{(o)}$ is the module's output; with history domains, several signal species map to the same formal species. For signal reversal (Figure 10) it adds the mappings $\forall j \neq i$: $\langle r_i\ t_r\ p_j \rangle \mapsto S_i^{(r)}$.[7]

Bisimulation theory requires fuel to be removed from the implementation CRN before verification. Because the definition of fuel in [17] does not fully match its use in classifier translation schemes [3, 4], we can instead treat supporting species as formal ones, e.g. representing annihilation by $C_i + C_j + Anh_{ij} \to \varnothing$, and add their mappings to the partial interpretation.

Proving bisimulation equivalence is NP-complete in the easiest cases [17], so we avoid proving each cascade separately. Ignoring crosstalk between species of different modules, formal correctness of two modules implies correctness of their composition [17, Theorem 3.2], so any cascade can be verified from its parts. This also requires modularity of implementations with respect to shared species [17, Definition 3.3], which is trivially satisfied here: the only shared implementation species are signal species, which by construction map to the formal *signals*, the only shared formal species. The same holds if supporting species are made formal, since none is shared by two modules.

2.4. Simulation. Initial conditions of supporting species are fixed by the translation scheme; the user sets input concentrations. For 2-input WTA we pass a vector of continuous values (e.g. $[0.8, 0.3]$). For a binary input $\boldsymbol{x}$ we set $[X_i]_0 = 0$ if $x_i = 0$ and $[X_i]_0 = Z$ if $x_i = 1$, where $Z > 0$ is arbitrary, since a catalytic cycle of two seesaw reactions runs at any non-zero input concentration [7]; larger Z shortens the response time. The simulation yields time courses of output concentrations. Because some operations in the classifiers include reversible reactions and their outputs may oscillate, we append a reporting module to the cascade, which is anyway needed for *in vitro* validation.

[7] When reversal is directly followed by annihilation, the output toehold t_r consists of two domains; in Figure 10, $t_r = \langle T\ s \rangle$.

We simulate deterministically: the CRN becomes a system of ordinary differential equations (ODE) governed by mass-action kinetics,

$$\frac{d[X]}{dt} = \sum_{r \in CRN} (\beta_X^r - \alpha_X^r)\rho(r), \tag{1}$$

where α_X^r (β_X^r) is the stoichiometric coefficient of X as a reactant (product) in reaction r and $\rho(r)$ is the rate of r. The system is integrated numerically from the initial concentrations.

3. Evaluation methodology

With simulations we tried to reproduce *in vitro* results from the translation scheme authors [3, 4], mimicking the original initial conditions as closely as possible. We tested translation-scheme variants for individual components and complete classifiers, and evaluated some parameter variations not evaluated in the original papers.

3.1. Metrics. Beyond qualitative correctness (which, for a suitably designed formal CRN, follows directly from formal correctness), we consider circuit size, verification complexity, state separation and completion time.

Synthesis complexity is the size of the prepared DSD system, which mainly determines synthesis cost. It could be measured by the number of complexes or by total domain length; we count complexes.

Verification complexity is the difficulty of *in silico* verification. We measured the time for reaction enumeration and CRN condensation (execution of the translation scheme is negligible). Timings were taken on a PC with an AMD Ryzen 7 5800X3D at default clock, 32 GB DDR4-3600, under Windows 11 Home (24H2, build 26200), running Python 3.12.4. As runtime is hardware dependent, we also use the size of the implementation CRN as a universal measure, given as a pair (s, r) of the number of species s and number of reactions r. Species in detailed CRNs are complexes; in condensed CRNs they are resting macrostates.

State separation: for WTA$(0.8, 0.2)$, say, the first output should quickly reach a high standard concentration (meaning output value 1) while the second stays near zero.

Completion time. Concentration trajectories are nonlinear and asymptotically approach the theoretical maximum, so we use the p-completion time t_p, the time at which a product reaches a fraction p of its maximum ($0 < p < 1$); we use $t_{95\%}$. Reporting is irreversible, so product trajectories are increasing and t_p can be computed by binary search.

3.2. Limiting molecule size. Besides timescale separation, `Peppercorn` allows an upper bound v_{max} on the number of strands per complex; any reaction producing a product with more than v_{max} strands is discarded. Correct modelling of WTA and LTA needs $v_{max} \geq 4$, as four strands are joined in the second annihilation phase (two signals and two annihilator strands). Smaller v_{max} lowers verification complexity but yields an implementation CRN less consistent with `Peppercorn`'s biophysical model. Reducing v_{max} from h to l costs negligible accuracy if secondary structures with more than l and at most h strands are unstable: if all complexes above l strands had sufficiently high decay rates at $v_{max} = h$, their maximum concentrations in simulation would be negligible.

3.3. Evaluating classifiers. Test cases should sample the space of weighted sums $\boldsymbol{x}^T\boldsymbol{W}$ uniformly, as experimental feasibility of WTA and LTA depends on the mutual similarity of the weighted sums [3, 4]. LTA is evaluated on harder cases than WTA, since negative classification is meant for inputs that are harder to separate. We are interested in the robustness of the molecular implementation for a fixed $\boldsymbol{W}$ under various translation-scheme parameters, so we did not compute classification accuracy. As in [3, 4], we tested the circuits on handwritten digits from MNIST.

4. Results

4.1. WTA function. For a unique maximum v_i in $\boldsymbol{v}$, the mathematical WTA outputs $o_i = 1$ and $o_{j \neq i} = 0$. In the circuits the output vector is shown by limiting concentrations of the output signals O_j: ideally $[O_i]_\infty = 1$ and $[O_{j \neq i}]_\infty \ll 1$. For 3-WTA (Figure 11a) with input $[0.05, 0.9, 0.4]$ we observe the correct asymptotic behaviour $[O_2]_\infty = 1$ (Figure 11b). Because the third input (0.4) is not very small compared with the maximum (0.9), we also observe a high noisy activation of the third output (green).

We compared two variants of the translation scheme for 2-WTA with inputs $[a, b]$ (Figure 12). They differ only in the length of toehold t_o, whose binding starts the signal-restoration catalytic cycle, by one nucleotide. Domain t_o is a prefix of t_k, which

starts annihilation. We set $|t_k| = 7$ as in the original design [3, 4]. We first tried $|t_o| = 5$ as originally designed (blue cells, $a > b$), and in the orange cells ($a < b$) a toehold one nucleotide shorter ($|t_o| = 4$). CRN condensation was used.

4.2. Signal reversal. Signal reversal can be used as a standalone operation or wired into WTA to implement LTA. For n signals the initial concentrations of the reversal gates must satisfy $[OG_{ij}]_0 \geq \frac{1}{n-1}$. Completion time is easiest to observe with inputs with a single non-zero value: with only one input X_i present, all non-zero outputs Y_j, $j \neq i$, have identical trajectories. We measured $t_{95\%}$ for reversal of three signals $[a, 0, 0]$ as a function of gate concentration and of $[X_1]_0 = a$ (Figure 13).

4.3. LTA function. Let X_i denote the species of the i-th LTA input, Y_j the reversed signals and O_j the outputs of restoring Y_j. The designed reactions are $X_i \to Y_j$, $Y_j + Y_k + Anh_{jk} \to \varnothing$ and $Y_j \to Y_j + O_j$. The LTA circuit works like WTA but yields $[O_i]_\infty = 1$ where v_i is the unique *minimum* of $\boldsymbol{v}$ (Figure 14).

Rodriguez *et al.* [4] observed that state separation in the *in vitro* implementation improved markedly when a new toehold was introduced at the 3′ ends of strands X_i. We tried to reproduce this with DISCO, setting all translation scheme parameters and initial conditions identically.[8] We nevertheless observed no difference in the time courses of the output signals.

We compared the effect of v_{max} on the verification complexity of 3-input LTA with a global toehold and with three local toeholds, one per signal X, Y, O (Table 1). The condensed CRN had the same size in all cases.

Table 1. Verification complexity of 3-LTA with global and local toeholds for various v_{max}, given as the size of the detailed CRN and its enumeration time. The condensed CRN is the same in all eight cases, of size $(345, 1089)$.

v_{max}	global toehold	local toeholds
4	$(1791, 5019)$, 23 s	$(675, 1941)$, 13 s
5	$(3105, 7935)$, 26 s	$(1611, 4101)$, 15 s
6	$(10035, 24315)$, 50 s	$(3195, 7845)$, 18 s
7	$(23103, 59955)$, 2 min 46 s	$(3195, 7845)$, 19 s

We compared the verification complexity of 4-LTA at $v_{max} = 4$ with different numbers of toeholds (Table 2). In row 1 signals X, Y, O share one toehold; in row 2 only X has its own toehold ($t_x \neq t_y = t_o$); in row 3 each signal has a local toehold ($t_x \neq t_y$, $t_x \neq t_o$, $t_y \neq t_o$).

Table 2. Verification complexity of 4-LTA at $v_{max} = 4$. The number of distinct toeholds increases down the rows. Second column: size of the detailed CRN and enumeration time. The condensed CRN has size $(1788, 6484)$ in all cases.

no. of toeholds	detailed CRN	condensation time
1	$(7590, 22492)$, 10 min 42 s	1 min 44 s
2	$(5286, 15100)$, 6 min 45 s	51 s
3	$(3046, 10044)$, 6 min 32 s	24 s

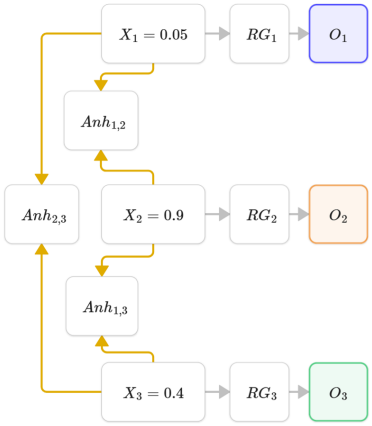
We also removed one domain from the 5′ ends of strands X_i (Figure 15). This makes signal reversal reversible and reduces state separation [4].

4.4. Classification circuits. We tested the WTA classifier on handwritten digits 6 and 7 (Figure 16). Input patterns have $n = 10 \times 10 = 100$ pixels, and the weight matrix $\boldsymbol{W} \in \mathbb{R}^{100\times 2}$ is the same as in [3]. We chose inputs from different parts of the weighted-sum space. Enumerating a CRN of size $(11189, 22096)$ with $v_{max} = 4$ took 27 s and condensation 2 min 11 s, so computing the condensed implementation CRN of size $(355, 262)$ took 2 min 38 s in total.

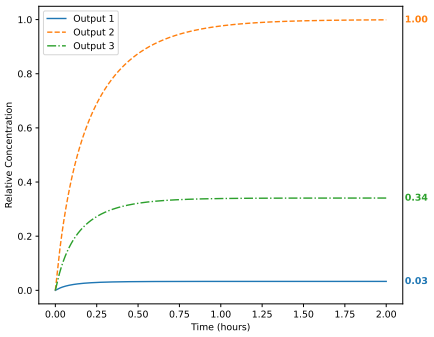
We tested negative classification of 100-bit patterns of digits 2, 3 and 4 with the weights from [3]. With $v_{max} = 4$ and local toeholds, enumeration did not finish within one hour and we stopped it. After removing domain T from the 5′ ends of the extended toeholds on the weight molecules (Figure 7), computing the condensed CRN of size $(577, 941)$ finished in 8 min 38 s (Figure 17a). This modification yields the reversible implementation of signal reversal, which has worse state separation (Figure 15).

To reduce verification complexity we also tried transforming the weight matrix and using the WTA circuit (Section 1.4). Computing the condensed CRN of size $(571, 599)$ took 13 min 31 s, of which 11 min 36 s was condensation. This approach raises synthesis complexity from 215 to 245 species, since $\boldsymbol{W} \to \boldsymbol{W}'$ densifies the matrix, but gives somewhat better state separation in the outputs (Figure 17b).

[8] Rodriguez *et al.* calibrated the concentrations of individual supporting species *in vitro* to reduce rate differences between species with different but equally long domains. `Peppercorn` computes rate constants from domain lengths only, not DNA sequences, so such differences cannot arise in the implementation CRNs computed by DISCO. We did not apply the calibration, as it would destroy the uniformity of the rates computed by `Peppercorn`.



(a) Diagram of the 3-WTA circuit. Yellow arrows mark faster reactions, mostly occurring before the others (grey). Reporter gates are not shown.



(b) Simulation results.

Figure 11. The 3-WTA circuit with input vector $[0.05, 0.9, 0.4]$.

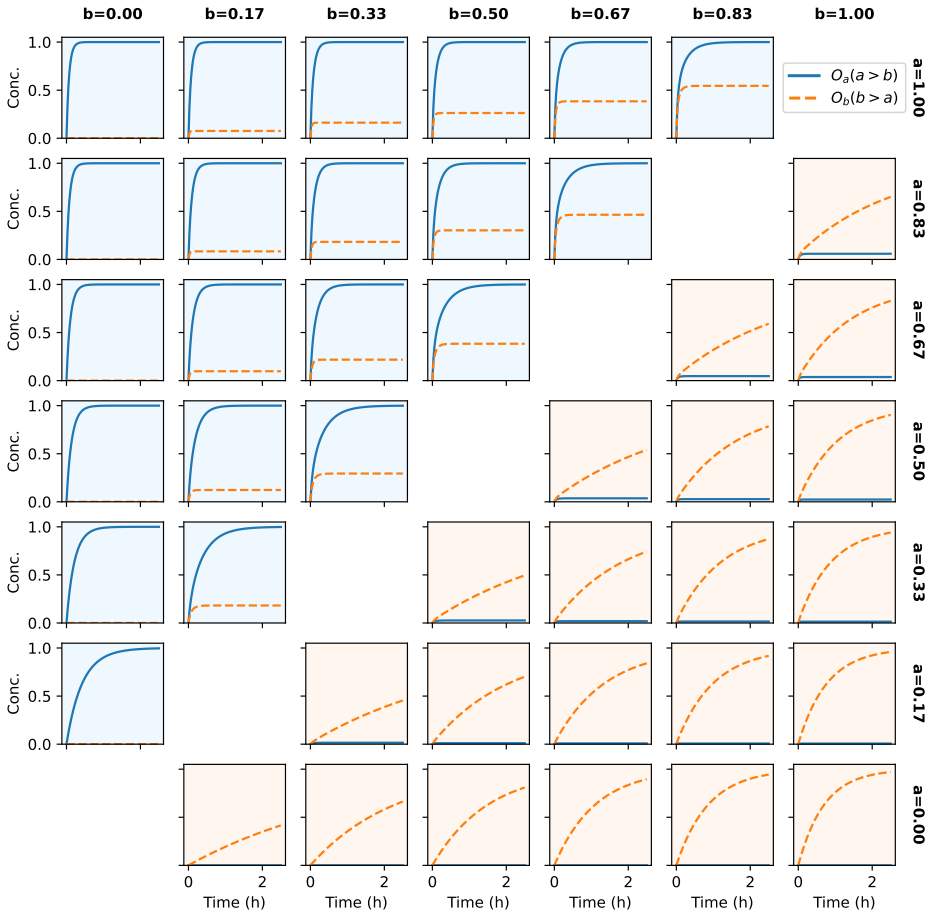


Figure 12. Simulation of the circuit computing WTA(a, b) for various input values. Upwards, a increases (right edge); rightwards, b increases (top edge). Each cell shows relative concentration (y) against simulation time in hours (x). The output O_a, corresponding to $a > b$, is the solid blue line; O_b $(b > a)$ is the dashed orange line. Diagonal cells $(a = b)$ are left empty as WTA is undefined there. Top left (blue background): variant $|t_o| = 5$; bottom right (orange background): variant $|t_o| = 4$. Showing the same scheme on both sides of the diagonal is pointless because WTA is symmetric.

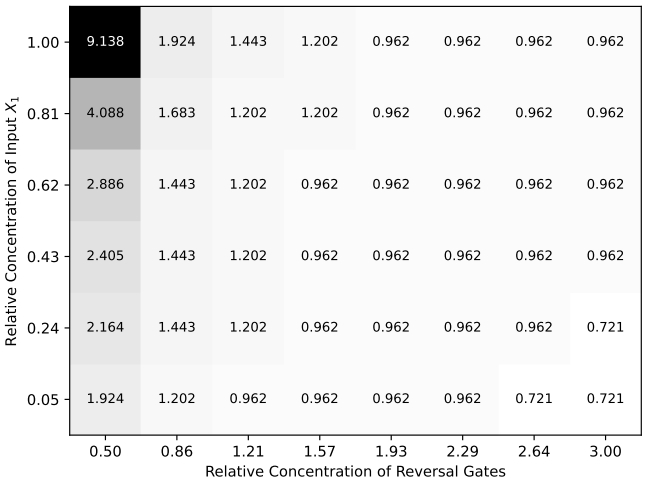
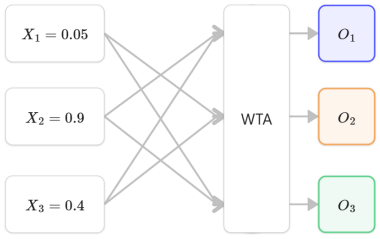
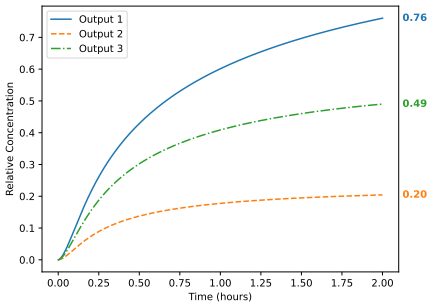


Figure 13. Completion time of the signal-reversal circuit in minutes. Upwards, input value a (relative concentration of X_1) increases; rightwards, the initial relative concentration of the reversal gates increases. For input $[a, 0, 0]$ only $X_1 + OG_{1,2} \to Y_2$ and $X_1 + OG_{1,3} \to Y_3$ occur, at equal rates. Each cell shows the $t_{95\%}$ at which Y_2 and Y_3 simultaneously exceed 95% of their final concentration $[Y_2]_\infty = [Y_3]_\infty = \frac{a}{n-1} = \frac{a}{2}$.



(a) Diagram of the 3-LTA circuit. Outputs of signal reversal Y_i feed the WTA circuit (Figure 11a).



(b) Simulation results.

Figure 14. The 3-LTA circuit with input vector $[0.05, 0.9, 0.4]$.

4.5. Formal verification results. We formally verified the implementation of each module separately (Table 3). For each module we built a circuit containing only that module, generated the formal CRN (column 2) and the initial partial interpretation (Section 2.3), computed the condensed implementation CRN, and checked CRN equivalence.

Table 3. Formal correctness of module implementations. Both the irreversible (row 1) and reversible (row 2) implementations of signal reversal were tested. Column 2: formal CRN without supporting species. Column 3: bisimulation equivalence [17]. Column 4: results of the compositional and integrated hybrids [18], which were identical.

Module	Formal CRN	Bisimulation	Hybrid
Signal reversal (irrev.)	$\forall i \neq j\colon X_i \to Y_j$	Yes	Yes
Signal reversal (rev.)	$\forall i \neq j\colon X_i \rightleftharpoons Y_j$	No	No
Pairwise annihilation	$\forall i < j\colon Y_i + Y_j \to \varnothing$	Yes	No
Signal restoration	$\forall i\colon Y_i \to Y_i + O_i$	No	No
Reporting	$\forall i\colon O_i \to Fluor_i$	Yes	Yes

Results were the same whether or not supporting species were included in the formal and implementation CRNs. They are reported only for condensed CRNs, as verifying detailed CRNs is mostly infeasible in time. We verified equivalence at signal dimensions d large enough for side reactions to occur; for pairwise annihilation, $d \geq 3$ ensures several different annihilators are present.

We could not prove correctness of signal restoration with one formal reaction per catalytic cycle ($Y_i + RG_i + F_i \to Y_i + O_i$, including supporting species), nor with two ($Y_i + RG_i \to O_i$ and $F_i \to Y_i$). Removing only the catalytic fuels F_i from the formal CRN, keeping the other supporting species (RG_i), did not change the result.

5. Discussion

5.1. Effect of translation-scheme parameters.

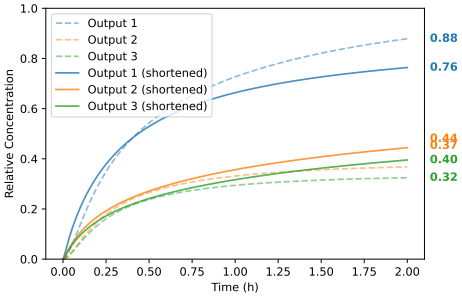


Figure 15. Effect of shortening input strands X_i on state separation in the LTA circuit. Dashed trajectories: standard input strands $\langle t_y \ s \ x_i \ t_x \rangle$; solid: shortened strands $\langle s \ x_i \ t_x \rangle$. Input $\boldsymbol{x} = [0, 0.9, 1]$, as in the same experiment of Rodriguez *et al.* [4]. Simulation of the detailed CRN is shown; it is almost identical to that of the condensed CRN.

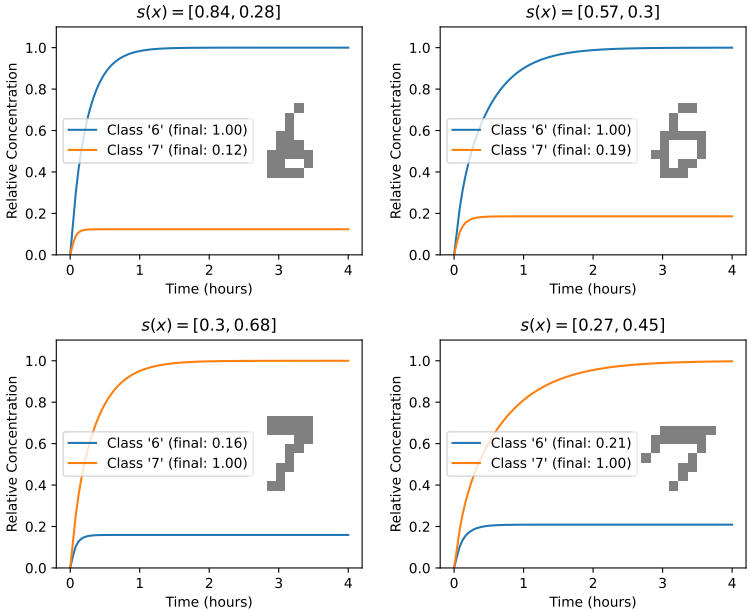


Figure 16. Output of the WTA classifier for handwritten digits 6 and 7, using four inputs from [3]. Above each graph are the weighted sums $s_i(\boldsymbol{x})$ for classes 6 and 7.

WTA circuit. For WTA(a, b) we compared the original scheme [3] (blue cells in Figure 12) with an alternative (orange). In the original scheme state separation is poor when $a \approx b$ (near the diagonal), especially when both arguments are large. This is because annihilation and restoration run in parallel [3]: a large share of signal $\min\{a, b\} = b$ escapes the annihilator and reaches the restoration gates, causing noisy activation of O_b. The difference in toehold lengths t_k (annihilators) and t_o (restoration gates) speeds annihilation relative to restoration, only approximating sequential execution.

State separation can thus be improved by a variant with larger $|t_k| - |t_o|$: the original has $7 - 5 = 2$; we shortened t_o from 5 to 4 nt, giving $7 - 4 = 3$. With $|t_o| = 6$ the approximation of sequential execution would be too weak ($|t_k| - |t_o| = 1$). We kept $|t_k| = 7$, since $|t_k| < 7$ further limits $|t_o|$ and complicates control over reaction order, while $|t_k| > 7$ risks a strong effect of occlusion of t_k. The variant with the larger difference (orange cells) has much better state separation: O_a stays low for $b > a$. However, completion time is much longer, because binding to the shortened t_o has a lower rate constant. This is the slowest bimolecular reaction in the circuit and thus the overall rate-limiting step. Since we used CRN condensation, t_k is the only toehold whose occlusion is present in the simulations.

Signal-reversal circuit. Figure 13 shows the effect of reversal-gate concentration on completion time. We used $n = 3$, which permits the least demanding experimental demonstration; for larger n the differences between reversed signals shrink [4], hampering the subsequent WTA [3].[9] Completion time is longer for high input values (top rows), since more input species must react with the gates, and we expect longer times for inputs whose values differ less, in both reversal and the subsequent WTA [3, 4]. Gate concentrations above the minimum $\frac{1}{n-1}$ speed up reversal (right columns), but may allow excessive progress

[9] Reversal of two signals ($n = 2$) is pointless, since the same operation is achieved by swapping two links in the circuit topology.

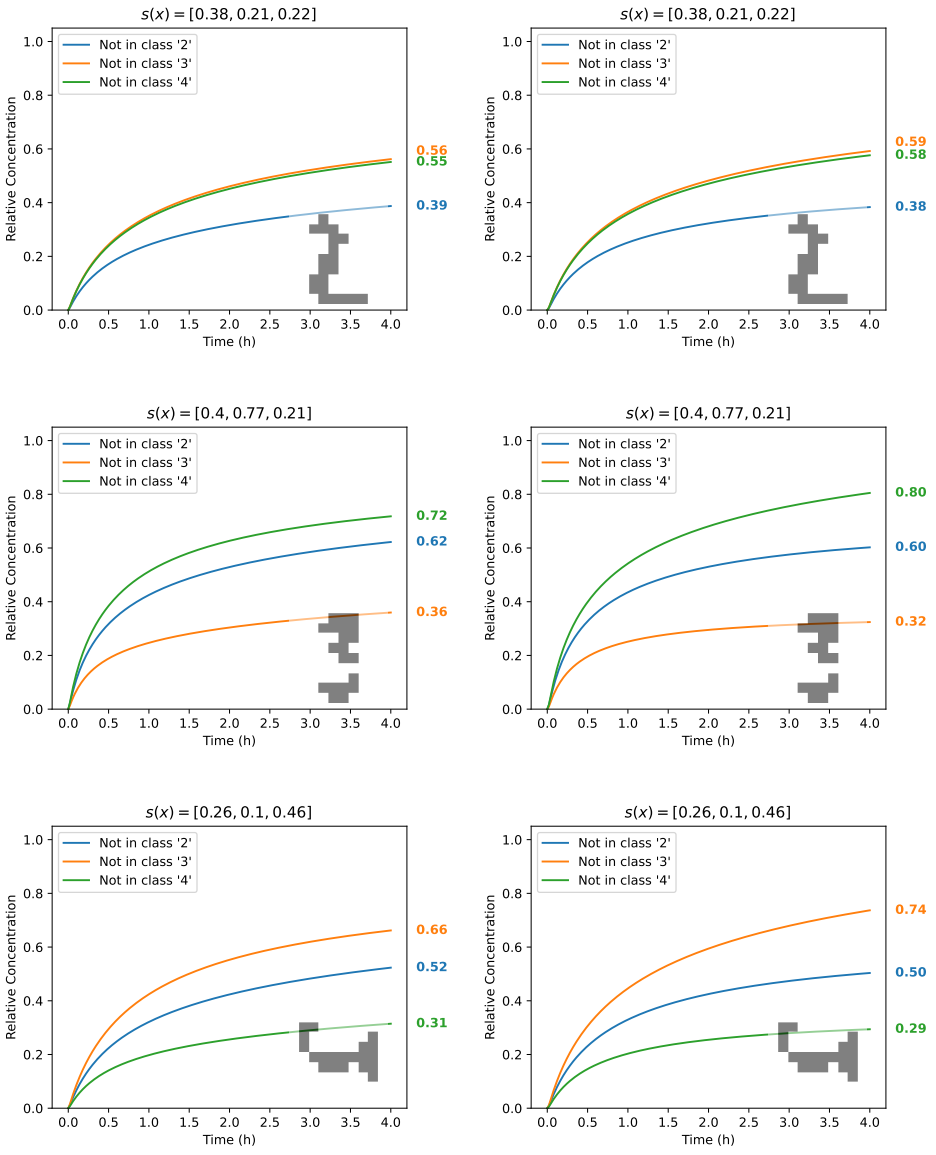
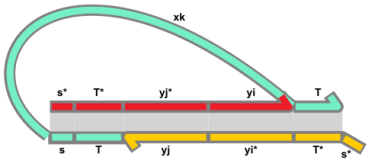


Figure 17. Output of the LTA classifier for handwritten digits 2, 3 and 4. Left column (a): LTA circuit with reversible signal reversal; right column (b): WTA circuit with transformed weights. Above each example $\boldsymbol{x}$ are the weighted sums $s_i(\boldsymbol{x})$ for classes 2, 3, 4. Negative classification is correct in all cases, as the i-th output is never the largest for $\boldsymbol{x} \in C_i$.

of some reactions (but not others) owing to rate variance [4], potentially reducing *in vitro* robustness. We could not evaluate this with DISCO, since such variance stems from sequence differences that `Peppercorn` ignores; Rodriguez *et al.* [4] did not test or mention this effect.

LTA circuit. In Figure 15 we compared LTA with irreversible and reversible signal reversal. We successfully reproduced the experimental results of [4]: owing to the inverse seesaw reactions $Y_j \to X_i$, state separation degrades substantially. Results are nearly identical with or without condensation, since the offending reactions are standard seesaw reactions that condensation does not remove.

We did not manage to reproduce the improvement in LTA state separation from a local toehold for input signal X. Rodriguez *et al.* [4] hypothesised that LTA circuits with a global toehold have worse state separation because of double occlusion between annihilators Anh_{jk} and inputs X_i; call the product of such an occlusion P_d (Figure 18a). A new toehold at the 3′ ends of strands X_i prevents formation of P_d.



(a)

(v, j)	$\pi(v, j)$
$(A_b, 2)$	$(X, 4)$
$(A_b, 3)$	$(A_t, 4)$
$(A_b, 4)$	$(A_t, 3)$
$(X, 1)$	$(A_t, 2)$
$(X, 2)$	$(A_t, 1)$

(b)

Figure 18. Product of double toehold occlusion between annihilator Anh_{ij} and input signal X_k (a) and its tabulated secondary structure (b). Each row is a pair of hybridised domains, labelled (v, j) with v the strand and j the domain index. The light-blue signal strand is X, the lower (yellow) annihilator strand A_b, and the upper (red) one A_t.

With the default k_{fast} (Section 2.2), P_d is a transient complex, since unbinding of the shorter toehold T is fast. To reproduce the results we therefore could not use CRN condensation. For manageable verification complexity of the detailed CRN we reduced v_{max} to the minimum permissible value, 4 (Section 3.2). As P_d has three strands, this did not prevent its formation.

If the double-occlusion hypothesis is correct, the failed reproduction can be explained by showing that DISCO cannot compute the reaction producing P_d. `Peppercorn` supports only pseudoknot-free secondary structures [20]. We show that P_d contains a pseudoknot, so `Peppercorn` cannot predict double occlusion.

A secondary structure π is pseudoknot-free if and only if there exists an ordering of strands $s_1, \ldots, s_n$ such that for every two hybridised pairs $\pi(s_i, j) = (s_k, l)$ and $\pi(s_p, q) = (s_u, v)$,

$$(i, j) < (p, q) < (k, l) \implies (i, j) < (u, v) < (k, l), \tag{2}$$

where $<$ is the lexicographic order [20, Definition 2.3]. In other words, if domain (s_p, q) lies between the domains of another hybridised pair, its partner must lie between the same domains, so every duplex splits the complex into two independent parts. For a pseudoknot-free structure exactly one circular ordering of strands[10] satisfies condition (2), since it is invariant under cyclic shifts [26, Theorem 2.1]. The presence of a pseudoknot can therefore be proved by brute force: write the structure of P_d as the table π (Figure 18b) and check condition (2) for every circular ordering. Doing so, we proved that P_d contains a pseudoknot. For a structure of $|\pi|$ hybridised pairs on n strands the computation takes $O\big((n-1)!\,(n+|\pi|+\binom{|\pi|}{2})\big) = O\big((n-1)!\binom{|\pi|}{2}\big)$, since $|\pi| \geq n-1$ for connected complexes. The condition can also be checked visually [26].

To model double occlusion with DISCO we would have to add reactions forming and dissolving all instances of P_d to the enumerated CRN by hand. From a biophysical viewpoint, however, good rate-constant approximations for such reactions are hard to compute, since pseudoknots have a complex energy landscape [27, 28]. Owing to the lack of suitable thermodynamic models, modelling tools such as `Peppercorn` ignore formation of structures containing pseudoknots.

5.2. Verification complexity.

Effect of toeholds. Besides circuit behaviour, some scheme parameters affect verification complexity. Tables 1 and 2 show that more distinct toeholds shrink the implementation CRN. Shared toeholds increase the number of possible occlusions; several successive occlusions can form polymers, and the resulting combinatorial explosion strongly affects verification complexity.

[10] A circular ordering is a permutation without start or end; a set of n objects has $(n-1)!$ of them.

Counting the longest polymers gives an asymptotic lower bound on verification complexity as signal dimensions grow. Write polymers as $Z_1 \circ Z_2 \circ \cdots \circ Z_p$, where a circle denotes unproductive binding (occlusion) of toeholds between neighbouring complexes. In particular, the open circle, $\circ$, denotes occlusion of a toehold that is sufficiently short that its unbinding is fast; call such toeholds *short*. Call the other toeholds, whose unbinding is slow, *long* toeholds, and denote their occlusion with a full circle as in $Z_i \bullet Z_{i+1}$. Occlusions of long toeholds give resting complexes (Section 1.5). With parameters suitable for LTA (Section 2.2), the compound toeholds $\langle T\ s \rangle$ and their complements are the only long toeholds in the circuit.

When both sides of X_i in a LTA circuit carry the same domain, i.e. $X_i = \langle T\ s\ x_i\ T \rangle$ (e.g. with a global toehold), we get polymers of the form $Anh_{ij} \bullet X_b \circ Anh_{kl}$, in which two annihilators are joined via a signal strand. First Anh_{ij} and X_b join via the long toehold $\langle T\ s \rangle$; since this is a resting complex it can take part in bimolecular reactions, and via the short toehold T on X_b it also binds Anh_{kl}. For $v_{max} \geq 7$ an even longer polymer forms. Denote $Y_{i \leftarrow j} = \langle y_i\ T\ s\ x_j \rangle$, a signal strand for formal signal Y_i with history domain x_j. These give can form complexes $Y_{a \leftarrow b} \bullet Anh_{ij}$ and $Anh_{kl} \bullet Y_{c \leftarrow d}$, which combine into the 7-strand complex

$$Y_{a \leftarrow b} \bullet Anh_{ij} \bullet X_e \circ Anh_{kl} \bullet Y_{c \leftarrow d}.$$

This is the largest complex we found at $v_{max} = 10$. For n-dimensional signals with $v_{max} \geq 7$ there are $\Theta\left(n^2 \times \binom{n}{2} \times n \times \binom{n}{2} \times n^2\right) = \Theta(n^9)$ such polymers, so the total number of implementation species is $\Omega(n^9)$; because of the history domains on $Y_{i \leftarrow j}$ it grows faster than $O(n^{v_{max}}) = O(n^7)$. For $n = 3$ with a global toehold (Table 1, last row) the estimate is fairly close to the actual number of species: $3^9/23103 \approx 85\%$.

All complexes with two annihilators are transient, since their formation ends with binding of a short toehold. They are therefore absent from the condensed CRN, yet their formation slows enumeration at $v_{max} \geq 5$. With the reversible implementation of signal reversal, where one domain is removed from the $5'$ ends of strands X_i, such polymers do not appear.

The reaction enumerator assumes a potentially unlimited number of instances of each molecule, so even at fixed synthesis complexity infinite polymerisation can occur, and verification complexity would grow without bound with increasing v_{max}. Species with at least two long toeholds are the only ones that can link ever longer resting polymers, via the pathway[11]

$$A_1 \bullet \cdots \bullet A_p + B + C_1 \bullet \cdots \bullet C_q \rightarrow A_1 \bullet \cdots \bullet A_p \bullet B \bullet C_1 \bullet \cdots \bullet C_q$$

where B is the linking species. In our circuits annihilators are the only such species with two long toeholds. As those two copies are not mutually complementary, annihilators cannot polymerise indefinitely; polymerisation is finite and verification complexity does not grow without bound as v_{max} increases.

We did not list condensed-CRN sizes in Tables 1 and 2 because they are identical across each table. Their independence from v_{max} within a column of Table 1 shows that in LTA circuits all complexes with more than 4 strands are transient, which to some extent justifies using $v_{max} = 4$ (Section 3.2). Equality of the condensed CRNs between global and local toeholds at fixed v_{max} shows that a global toehold merely allows additional transient complexes, which condensation removes; with local toeholds we lose no resting (stable) complexes that could substantially affect simulation results.

A local toehold at the $3'$ ends of strands X_i prevents annihilators from linking together. In Table 1 the detailed CRN with local toeholds is the same at $v_{max} = 6$ and $v_{max} = 7$, because complexes with more than 6 strands cannot form. The complex $Anh_{kl} \bullet Y_{c \leftarrow d}$ can still link via X_e with reversal gates into the polymer $OG_{ab} \circ X_e \bullet Anh_{kl} \bullet Y_{c \leftarrow d}$, even with a unique toehold per signal. This is transient, since the leftmost bond is a short toehold. For $v_{max} \geq 6$ there are $\Theta(n^2 \times n \times \binom{n}{2} \times n^2) = \Theta(n^7)$ such 6-strand polymers, so the total number of species is $\Omega(n^7)$. For $n = 3$ with local toeholds (Table 1, last column) this estimate accounts for $3^7/3195 \approx 68\%$ of the actual species at $v_{max} \geq 6$.

Limiting the number of molecules. Even relatively small circuits can exceed `Peppercorn`'s default upper limit on the number of implementation species, $c_{max} = 10^4$ (Table 1); condensation does not help, as it runs after enumeration. When setting c_{max} manually, the lower bounds above help: for n-LTA at $v_{max} \geq 7$ set $c_{max} > n^9$.

The weight-multiplication module with $\boldsymbol{W} \in \mathbb{R}_+^{b \times n}$ (for classifying b-bit patterns into n classes) further constrains c_{max}. At high input dimension b the default c_{max} can be exceeded even at $v_{max} = 4$, as it is for $b = 100$ (Section 4.4). One reason is the long toeholds $\langle T\ s \rangle$ on the weight molecules (Figure 7), which produce resting occlusion products; in Section 4.4 we fixed excessive enumeration time by removing a domain from them.

[11] We write a trimolecular reaction as shorthand for two successive bimolecular reactions.

To compute a suitable c_{max} we need the dependence on b. Even with a local toehold per signal, many occlusions among complexes within the multiplication module are possible. Let $B_{>0} \subseteq [1..b]$ be the set of row indices of $\boldsymbol{W}$ with positive sum, $|B_{>0}| = f \leq b$. The catalytic cycle in this module creates intermediates XW_k for each $k \in B_{>0}$ and P_{ki} for each non-zero weight $w_{ki} > 0$ [3]. Complexes XW_k have a free domain t_p^* and P_{ki} have t_p, where t_p is the toehold of the module's output signal, so (mostly unproductive) bindings $XW_a \circ P_{bc}$ occur. There are f species XW_a and at most fn species P_{bc}, so about f^2n transient complexes can appear, and suitable values of c_{max} are much larger than f^2n. A less precise bound valid for all $\boldsymbol{W} \in \mathbb{R}_+^{b\times n}$ is $c_{max} > b^2n$.

Alternative timescale separation. In Section 2.2 we derived parameters under which all (even condensed) implementation CRNs contain occlusions of the long toehold t_k. These permit additional polymers that limit DISCO's scalability (Table 1).

For products of occlusion over t_k to be marked transient, its unbinding would have to be fast. The reverse branch migration across c_i (from state `e5` to `e3` in Figure 8) would then have to be slow so that the second annihilation phase can occur; were it fast, `e5` would be transient and could not react bimolecularly with C_j. With fast unbinding of t_k, migration towards the middle of the annihilator (`e3` to `e5`) must also be fast: were it slow, the fast unbinding of t_k would occur first and even the first phase could not complete. If forward migration were faster than its inverse, $k_m^{(fwd)}(|c|) > k_m^{(rev)}(|c|)$, the criteria could in principle be met with parameters satisfying

$$k_{slow} \leq k_m^{(rev)}(|c_i|) < k_{fast} \leq \min\left\{k_u(|t_k|), k_m^{(fwd)}(|c|)\right\}. \tag{3}$$

However, `Peppercorn`'s rate-approximation model ignores context differences (here only one migration follows a prior toehold binding) and computes equal constants $k_m^{(fwd)}(\ell) = k_m^{(rev)}(\ell) = \frac{10^3}{3\ell}\,\mathrm{s}^{-1}$ [20]. Hence inequalities (3) cannot be satisfied regardless of domain lengths. Slow unbinding of t_k is therefore mandatory, its occlusions cannot be removed by condensation, and formation of the polymers described above cannot be prevented.

5.3. Formal correctness of implementations. In Table 3 we proved the correctness of some modules' implementations for bounded signal dimensions. Under certain assumptions (Section 2.3), correctness of individual modules also proves correctness of every possible cascade under bisimulation theory. If we succeeded in proving modularity of implementations [17, Definition 3.3] at the level of individual formal reactions *within* a module, we could prove correctness for all signal dimensions [17]. This would require ruling out crosstalk between implementations of different formal reactions within one module, or simply choosing to ignore its effect for the purpose of formal verification. This is harder to justify at the intra-module level than the inter-module level, since many intra-module occlusions can occur even with local toeholds. In our case, modularity is applicable only to some condensed implementation CRNs, as all detailed CRNs contain occlusions (note that occlusions of long toeholds are present also in condensed CRNs).

The reversible implementation of signal reversal (Figure 15, solid trajectories) is not equivalent to the reversible formal reactions $X_i \rightleftharpoons Y_j$ (Table 3, row 2). Bisimulation requires an interpretation of implementation species I_s that induces a *correct* interpretation $I_r\colon R' \to R$ of implementation reactions into formal reactions, i.e. one satisfying all three bisimulation conditions [17]. The problem is the *permissive condition*: every formal reaction must be mimicked by at least one implementation reaction pathway. For each formal reaction r possible from formal state S, from every implementation state S' with $I_s(S') = S$ there must be a sequence of reactions z with $I_r(z) = r$. This is impossible for the inverse of the implementation reaction $x_i + OG_{ij} \to y_j + p$, whatever the interpretation of the by-product p: from the implementation state $\{y_j\}$, interpreted as $\{Y_j\}$, in the absence of p no inverse reaction can mimic the formal inverse reaction $Y_j \to X_i$.

For a similar reason this implementation is incorrect under hybrid equivalence too. Both algorithms (compositional and integrated) first check that the implementation CRN is *tidy* [18]. The reaction $x_i + OG_{ij} \to y_j + p$ is not tidy, since p cannot be consumed without simultaneously consuming the species implementing a formal species (here Y_i).

In case of a negative result the hybrid methods also output a pathway of implementation reactions inconsistent with the formal CRN. For pairwise annihilation we found toehold occlusions in all such pathways. This is the only module with long toeholds, so even its condensed CRN contains occlusions that violate the hybrid-equivalence conditions.

Because of occlusions, this module's implementation is not modular at the level of individual reactions [17]. Its correctness by bisimulation (Table 3, column 3, row 3) therefore cannot be generalised to arbitrary dimensions, regardless of whether annihilators are included in the formal CRN.

We could not prove correctness of signal restoration with any of the methods used (Table 3, row 4). We expect the same for weight multiplication, as it is implemented

identically. As signal restoration is formally incorrect, we expect WTA, the cascade of annihilation and restoration, to be incorrect as well. We nonetheless tried to verify the implementation CRN of WTA computed with rate-dependent timescale separation (Section 2.2), which was not used for restoration alone, and still obtained a negative result: the WTA implementation is formally incorrect. Because of the formal incorrectness of signal restoration we cannot verify more complex circuits containing it; by bisimulation and hybrid equivalence the implementations of WTA, LTA and the classification circuits are incorrect. Proving rate-independent equivalence of the formal and implementation CRNs is in any case not a very good verification of the WTA translation, since its operation depends strongly on relative rates (for the effect on state separation see Figure 12).

5.4. Comparison of DISCO with existing tools. With DISCO we found secondary structures and reactions that other tools (e.g. `Visual DSD`) cannot find (Section 5.2). In an LTA circuit with the same toehold on the input and output of signal reversal ($t_x = t_y$), even at the strictest molecule-size limit $v_{max} = 4$, `Peppercorn` predicts remote strand displacement (RTMSD) and monomolecular binding (Figure 19).

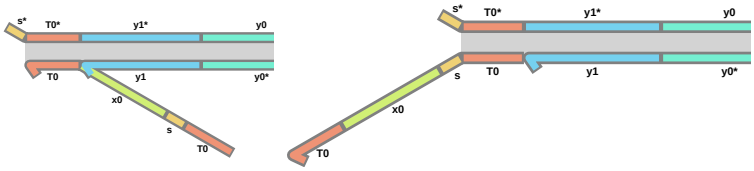


Figure 19. Reversible RTMSD in which strand X_0 on complex $X_0 \circ Anh_{0,1}$ displaces itself. The left state is the first that can be reached. The right state cannot be initial under max-helix semantics, as domains s and $T0$ would bind simultaneously; it is reached only via RTMSD. As $T0$ is a short toehold, both complexes are transient, so the whole pathway can be removed by condensation. From the right state monomolecular binding of domain s is also possible, similar to Figure 5.

As the RTMSD reactions in Figure 19 occur between transient complexes, they do not significantly affect circuit operation. The rate in both directions is $k \approx 1.68\ \mathrm{s}^{-1}$. With the parameters of Section 2.2, $k > k_{fast} = k_u(7) \approx 0.098$, so both are classified as fast. This does not necessarily mean they are influential, since the split of monomolecular reactions into fast and slow is relevant only to the enumeration process; the constant is relatively small compared with other monomolecular reactions (Figure 9).

We never encountered four-way branch migration, so by using the enumerator we have indirectly shown that such undesired reactions, assuming orthogonal domains, are not possible under the *in vitro* conditions that `Peppercorn`'s biophysical model captures well. This could not be shown with `Visual DSD`, even if the annihilation modelling problem were solved, as its enumerator does not support that reaction.

As `Visual DSD` has no biophysical model for approximating rate constants, binding and unbinding constants for toeholds must be assigned manually. In some cases this is an advantage, as it allows modelling phenomena arising from concrete DNA sequences; Rodriguez *et al.* [4], for example, observed strong variance in the rates of parallel annihilation reactions, which forced them to calibrate the concentrations of individual annihilators. Such phenomena are invisible in DISCO simulations because `Peppercorn` computes constants from domain lengths alone.

One side reaction that `Peppercorn` does not support is *zero-toehold branch migration*, in which migration occurs without prior toehold binding [12]. As `Peppercorn` cannot enumerate it, DISCO cannot confirm the absence of such reactions or the negligibility of their effect. The seesaw compiler models some zero-toehold migrations with rate constants obtained from measurements [12]. In assembling the DSD system (Section 2.1) we omitted *clamp domains* [4], as these serve only to prevent zero-toehold migration.

The seesaw and WTA compilers always simulate certain undesired reactions, whereas `Peppercorn`, via CRN condensation and filtering of selected reaction types, leaves that decision to the user. Exhaustive enumeration can, however, produce an implementation CRN far too large for practical analysis; this is why, in Section 4.4, we could not evaluate the standard translation scheme for the 100-bit LTA classifier with DISCO. The seesaw and WTA compilers avoid the complexity of enumerating and simulating many occlusions with *lumped species optimisation* [7], which is somewhat like CRN condensation but projects the CRN along different "dimensions".

The `Nuskell` framework [16] enables a pipeline similar to DISCO for arbitrary DSD circuit architectures, integrating an interpreter for a translation scheme definition language (also called `Nuskell`), the `Peppercorn` enumerator and the verification methods of `crnverifier`. We implemented our own framework for assembling circuits and translating them into DSD systems so that arbitrary parameters can enter the translation scheme

(Figure 6), which the `Nuskell` language does not allow. `Nuskell` scripts take a formal CRN as input, whereas with DISCO one explicitly assembles (supported) mathematical operations instead of writing formal CRNs; the formal CRN is generated when needed to verify equivalence with the implementation CRN. `Nuskell` also cannot express the dependence between the translation scheme and the reaction enumeration parameters that DISCO provides by passing metadata through the pipeline stages. For pairwise annihilation this is essential, as the permitted ranges of timescale-separation parameters depend on the domain lengths on the annihilators (Section 2.2).

5.5. Future work. Building on top of existing DISCO features, one could iteratively optimise translation scheme parameters (domain lengths, relative concentrations, etc.) for good output characteristics, e.g. minimal completion time with bounded concentrations[12] or high state separation, for instance by evolutionary optimisation or differentiable programming on indexed CRNs [29].

One could also optimise classification accuracy (CA) for a fixed weight matrix $\boldsymbol{W}$. For fixed $\boldsymbol{W}$ the CA on a given validation set is theoretically constant, but the probability of correct classification in an *in vitro* implementation depends on experimental conditions. Under idealised conditions a WTA circuit classifies input $\boldsymbol{x}$ into class C_i if and only if the weighted sum for that class is largest. Cherry and Qian [3], however, computed *experimental* CA, where a classification counts as correct only if $\boldsymbol{x} \in C_i$ and $s_i(\boldsymbol{x}) = \boldsymbol{x}^T \boldsymbol{W}_{:,i}$ exceeds all other weighted sums by at least $\delta > 0$. The authors [3, 4] used a fixed function to approximate experimental CA, although it in principle depends on some scheme parameters and laboratory conditions. If experimental CA could be computed systematically for a DSD system produced by a specific scheme variant, translation parameters could be iteratively optimised to maximise it for fixed $\boldsymbol{W}$.

For comprehensive classifier design support, DISCO could also model the protocol for *in vitro* weight training: Cherry and Qian [30] developed a protocol for supervised learning and activation of weights in WTA classification circuits.

6. Conclusion

We developed DISCO[13], a tool for assembling circuits that process molecular signals and for *in silico* validation of their DNA strand displacement implementations. It currently supports only the operations needed to implement the WTA and LTA synthetic biological classifiers [3, 4] with the seesaw architecture [7].

Compared with existing tools specialised for simulating such circuits [3, 7], DISCO allows experimentation with a wide range of parameters. Its modular, parametrised design of translation schemes permits evaluating their modifications and optimisations. Integrating the `Peppercorn` enumerator [20] allows detailed modelling of the dynamics of the generated DSD system, and integrating formal methods [17, 18] allows verifying its conformance to an abstract specification of the circuit's intended behaviour.

We compared translation-scheme variants and qualitatively reproduced some *in vitro* results from the literature [3, 4]. We could not reproduce the difference in LTA state separation from using a local toehold instead of a global one. A possible explanation is *in vitro* formation of a pseudoknotted secondary structure, which `Peppercorn` cannot enumerate.

We estimated the asymptotic growth of the number of implementation species with circuit dimensions, and showed that the conditions for correctly modelling pairwise annihilation inevitably produce some transient complexes that substantially increase the time complexity of *in silico* verification. We implemented a scheme assigning local toeholds to individual signals, which reduces verification complexity without materially affecting simulation results.

Excessive enumeration time prevented us from evaluating some translation schemes with DISCO. This exposes a key limitation of explicit reaction enumeration: although seesaw circuits contain many structurally similar species, the enumerator treats them separately. Scalable analysis would need an implicit representation of combinatorial processes instead of explicit computation of the implementation CRN [31, 32].

A rate-free formal CRN may also not be the most suitable specification language for formally verifying the circuits considered. Checking CRN equivalence failed to formally prove the correctness of some operations, even though simulations confirm that they work in practice; at sufficiently high concentrations, implementation paths inconsistent with the formal CRN often have no significant effect [18]. Describing the desired circuit behaviour in a specification language that accounts for relative reaction rates might allow formal proofs for kinetics-dependent implementations such as the WTA translation scheme [3].

[12] Without concentration limits, the p-completion time can be shortened arbitrarily by raising the concentrations of supporting species.
[13] https://github.com/andrejjocic/displacement-cascade-compiler

Bibliography

1. Groves B et al. (2016) Computing in mammalian cells with nucleic acid strand exchange. *Nature nanotechnology* 11(3):287–294.
2. Sarraf N (2023) *Towards integrated molecular machines: structural, mechanical, and computational motifs*. (California Institute of Technology).
3. Cherry KM, Qian L (2018) Scaling up molecular pattern recognition with DNA-based winner-take-all neural networks. *Nature* 559(7714):370–376.
4. Rodriguez KR, Sarraf N, Qian L (2021) A loser-take-all DNA circuit. *ACS Synthetic Biology* 10(11):2878–2885.
5. Adleman LM (1994) Molecular computation of solutions to combinatorial problems. *science* 266(5187):1021–1024.
6. Liu Q et al. (2000) DNA computing on surfaces. *Nature* 403(6766):175–179.
7. Qian L, Winfree E (2011) A simple DNA gate motif for synthesizing large-scale circuits. *Journal of the Royal Society Interface* 8(62):1281–1297.
8. Grozinger L, Cuevas-Zuviría B, Goñi-Moreno Á (2025) Why cellular computations challenge our design principles in *Seminars in cell & developmental biology*. (Elsevier), Vol. 171, p. 103616.
9. Tang L, Luo T, Fan S, Liu Y, Song J (2023) Principles of nucleic acid toehold mediated strand displacement (TMSD) reaction model and its applications in cell environment. *Biomaterials Science*.
10. Qian L, Soloveichik D, Winfree E (2010) Efficient turing-universal computation with DNA polymers in *International Workshop on DNA-Based Computers*. (Springer), pp. 123–140.
11. Srinivas N, Parkin J, Seelig G, Winfree E, Soloveichik D (2017) Enzyme-free nucleic acid dynamical systems. *Science* 358(6369):eaal2052.
12. Qian L, Winfree E (2011) Scaling up digital circuit computation with DNA strand displacement cascades. *science* 332(6034):1196–1201.
13. Srinivas N et al. (2013) On the biophysics and kinetics of toehold-mediated DNA strand displacement. *Nucleic acids research* 41(22):10641–10658.
14. Lv H, Li Q, Shi J, Fan C, Wang F (2021) Biocomputing based on DNA strand displacement reactions. *ChemPhysChem* 22(12):1151–1166.
15. Zhang DY, Winfree E (2009) Control of DNA strand displacement kinetics using toehold exchange. *Journal of the American Chemical Society* 131(47):17303–17314.
16. Badelt S et al. (2017) A general-purpose CRN-to-DSD compiler with formal verification, optimization, and simulation capabilities in *DNA Computing and Molecular Programming: 23rd International Conference, DNA 23, Austin, TX, USA, September 24–28, 2017, Proceedings 23*. (Springer), pp. 232–248.
17. Johnson R, Dong Q, Winfree E (2019) Verifying chemical reaction network implementations: a bisimulation approach. *Theoretical Computer Science* 765:3–46.
18. Shin SW, Thachuk C, Winfree E (2019) Verifying chemical reaction network implementations: A pathway decomposition approach. *Theoretical Computer Science* 765:67–96.
19. Zadeh JN et al. (2011) Nupack: Analysis and design of nucleic acid systems. *Journal of computational chemistry* 32(1):170–173.
20. Badelt S et al. (2020) A domain-level DNA strand displacement reaction enumerator allowing arbitrary non-pseudoknotted secondary structures. *Journal of the Royal Society Interface* 17(167):20190866.
21. Johnson RF, Winfree E (2020) Verifying polymer reaction networks using bisimulation. *Theoretical Computer Science* 843:84–114.
22. Genot AJ, Zhang DY, Bath J, Turberfield AJ (2011) Remote toehold: a mechanism for flexible control of DNA hybridization kinetics. *Journal of the American Chemical Society* 133(7):2177–2182.
23. Lakin MR, Youssef S, Polo F, Emmott S, Phillips A (2011) Visual DSD: a design and analysis tool for DNA strand displacement systems. *Bioinformatics* 27(22):3211–3213.
24. Hucka M et al. (2008) Systems biology markup language (SBML) level 2: Structures and facilities for model definitions. *Nature Precedings* pp. 1–1.
25. Lakin MR, Youssef S, Cardelli L, Phillips A (2012) Abstractions for DNA circuit design. *Journal of The Royal Society Interface* 9(68):470–486.
26. Dirks RM, Bois JS, Schaeffer JM, Winfree E, Pierce NA (2007) Thermodynamic analysis of interacting nucleic acid strands. *SIAM review* 49(1):65–88.
27. Lyngsø RB, Pedersen CN (2000) RNA pseudoknot prediction in energy-based models. *Journal of computational biology* 7(3-4):409–427.
28. Isambert H, Siggia ED (2000) Modeling RNA folding paths with pseudoknots: application to hepatitis delta virus ribozyme. *Proceedings of the National Academy of Sciences* 97(12):6515–6520.
29. Lee I, Buse S, Winfree E (2025) Differentiable programming of indexed chemical reaction networks and reaction-diffusion systems in *31st International Conference on DNA Computing and Molecular Programming (DNA 31)(2025)*. (Schloss Dagstuhl–Leibniz-Zentrum für Informatik), pp. 4–1.
30. Cherry KM, Qian L (2025) Supervised learning in DNA neural networks. *Nature* pp. 1–9.
31. Kawamata I, Aubert N, Hamano M, Hagiya M (2012) Abstraction of graph-based models of bio-molecular reaction systems for efficient simulation in *International Conference on Computational Methods in Systems Biology*. (Springer), pp. 187–206.
32. Faeder JR, Blinov ML, Hlavacek WS (2009) Rule-based modeling of biochemical systems with BioNetGen in *Systems biology*. (Springer), pp. 113–167.